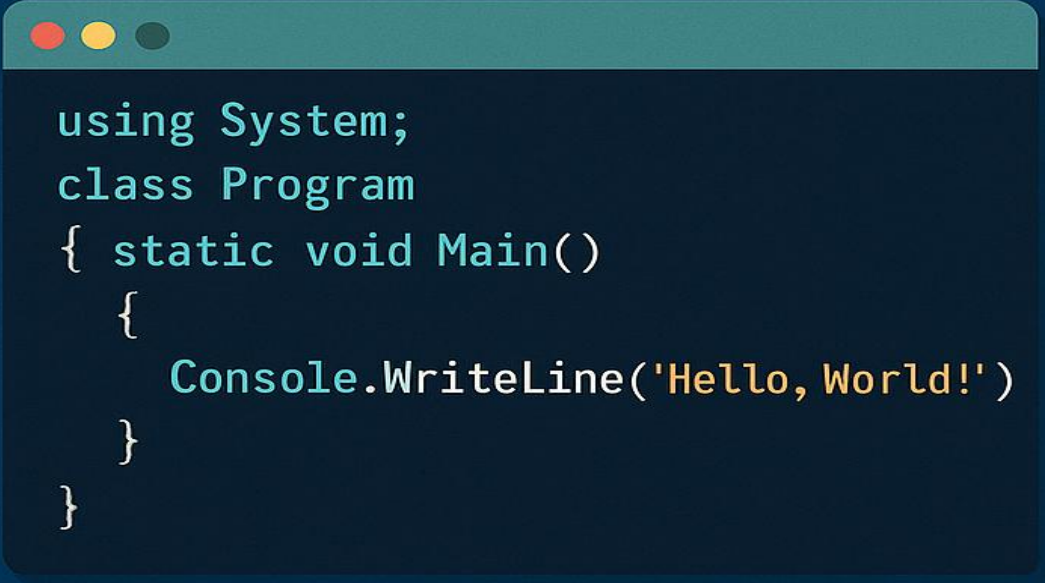


GLOSSÁRIO ABRANGENTE DE C#

PARA PROGRAMAÇÃO
ESTRUTURADA EM CONSOLE



```
using System;  
class Program  
{ static void Main()  
  {  
    Console.WriteLine('Hello, World!')  
  }  
}
```

INSTRUTOR:
DJALMA BATISTA BARBOSA JUNIOR

A Estrutura Fundamental de um Programa Console

Para iniciar a jornada na programação com C#, é imperativo compreender a estrutura basilar que sustenta qualquer aplicação de console. Embora possa parecer complexa à primeira vista, cada componente desta estrutura possui um propósito lógico e organizacional que se torna fundamental à medida que os programas crescem em complexidade.

A estrutura canônica namespace -> class -> Main não é um ritual arbitrário, mas uma hierarquia de organização deliberada. O namespace organiza as classes para prevenir conflitos de nomes em projetos de grande escala.

A class agrupa métodos e dados relacionados. O método Main fornece um ponto de entrada universalmente reconhecido pelo ambiente de execução do.NET. Esta arquitetura, embora pareça verbosa para um simples programa "Olá, Mundo!", é o que confere à linguagem C# a escalabilidade e robustez necessárias para o desenvolvimento de software complexo no mundo real.

1.1 A Diretiva using System;

No topo de quase todo arquivo de código C# encontra-se a diretiva using System;. Esta linha não é código executável, mas sim uma instrução para o compilador. Sua função é importar o namespace chamado System, que é uma vasta biblioteca de funcionalidades essenciais fornecidas pela plataforma.NET.

O namespace System contém classes e tipos de dados fundamentais que são utilizados em praticamente todas as aplicações. Entre os mais importantes para a programação em console estão a classe Console, que permite a interação com a janela do terminal (para escrever e ler texto), a classe Math, que oferece funções matemáticas, e os tipos de dados base como String e Int32 (o tipo por trás da palavra-chave int).

A principal vantagem de utilizar using System; é a simplificação do código. Sem esta diretiva, cada vez que se quisesse utilizar um recurso do namespace System, seria necessário escrever seu nome completo, conhecido como "nome totalmente qualificado". Por exemplo, para escrever uma linha no console, o código seria System.Console.WriteLine("Olá, Mundo!");. Com a diretiva using System; no início do arquivo, o compilador já sabe onde encontrar a classe Console, permitindo que o código seja escrito de forma muito mais concisa e legível: Console.WriteLine("Olá, Mundo!");. Esta simplificação é crucial para a clareza e para reduzir a carga cognitiva, especialmente para programadores iniciantes.

1.2 O Conceito de namespace

Um namespace em C# é um mecanismo de organização. Ele funciona como um contêiner ou um envelope que agrupa tipos relacionados, como classes, structs, interfaces e outros namespaces. Para um iniciante, uma analogia eficaz é pensar em um namespace como uma pasta em um sistema de arquivos ou como um "sobrenome" para os tipos de dados que ele contém.

O propósito principal de um namespace é duplo: organização e prevenção de conflitos de nomes. Em projetos pequenos, seu valor pode não ser imediatamente óbvio. No entanto, em sistemas grandes, é comum que diferentes partes do software, ou até mesmo bibliotecas de terceiros, definam classes com o mesmo nome.

Por exemplo, pode haver uma classe Lista no seu código de faturamento e outra classe Lista no seu código de gestão de pacotes. Ao colocar cada classe em um namespace diferente (por exemplo, Faturamento.Lista e Pacotes.Lista), evita-se a ambiguidade e o compilador sabe exatamente a qual classe se está referindo.

Mesmo que um namespace não seja declarado explicitamente em um arquivo, o compilador C# adiciona um namespace padrão, conhecido como namespace global. Qualquer identificador neste namespace está disponível para uso em outros namespaces nomeados. Embora não seja estritamente essencial para programas simples, declarar um namespace desde o início é uma prática recomendada que promove a organização do código.

1.3 A Estrutura de class Program

Dentro de um namespace, o próximo nível de organização é a class. Uma classe é um dos blocos de construção fundamentais em C#, servindo como um modelo para criar objetos que agrupam dados (variáveis) e comportamentos (métodos).

No contexto deste glossário, que se concentra na programação procedural e estruturada sem abordar a orientação a objetos, a class Program pode ser entendida de forma mais simples: é o contêiner obrigatório que abriga o ponto de entrada da aplicação, o método Main. Para que o programa possa ser executado, o método Main precisa estar contido dentro de uma classe (ou de uma struct). Por convenção, essa classe é frequentemente chamada de Program. A maior parte do código que os alunos escreverão em seus primeiros programas de console residirá dentro desta classe e, mais especificamente, dentro do método Main.

1.4 O Ponto de Entrada: `static void Main(string args)`

O método `Main` é o coração de uma aplicação de console em C#. É o ponto de entrada oficial, o primeiro método que é invocado quando o programa é executado, e é onde o controle do programa começa e termina. Só pode haver um único ponto de entrada em um programa C#. A assinatura `static void Main (string args)` é a forma mais tradicional e didática de declará-lo, e cada palavra-chave tem um significado preciso:

- **static:** O modificador `static` indica que o método `Main` pertence à classe `Program` em si, e não a uma instância específica (um objeto) dessa classe. Isso é crucial porque permite que o ambiente de execução do .NET chame o método `Main` sem precisar primeiro criar um objeto da classe `Program`, o que seria um paradoxo, pois a criação de objetos já é uma ação que ocorreria dentro do programa.
- **void:** Este é o tipo de retorno do método. `void` significa que o método `Main` não retorna nenhum valor para o processo que o chamou (geralmente, o sistema operacional). Em cenários mais avançados, `Main` pode retornar um `int` para indicar um código de status de saída (por exemplo, 0 para sucesso, um valor diferente de zero para erro), mas para a maioria das aplicações de console, `void` é suficiente e mais simples.
- **Main:** É o nome padrão e especial que o compilador C# procura para identificar o ponto de partida da execução. O nome deve ser exatamente "`Main`", com a primeira letra maiúscula.
- **(string args):** Este é o parâmetro do método. Ele declara uma variável chamada `args` que é um *array* (uma coleção) de strings (`string`). Este array é usado para receber argumentos passados para o programa através da linha de comando no momento da sua execução. Por exemplo, ao executar um programa como `meuprograma.exe arquivo.txt`, o array `args` conteria a string "`arquivo.txt`" em sua primeira posição (`args`). Embora não seja frequentemente utilizado em exercícios iniciais, é um mecanismo poderoso para tornar os programas mais flexíveis e interativos com o ambiente do sistema operacional.

É importante notar que versões mais recentes do C# introduziram as "instruções de nível superior" (*Top-Level Statements*), que permitem escrever código executável diretamente em um arquivo `.cs` sem a necessidade explícita de declarar a estrutura `namespace`, `class Program` e `Main`.

Isso demonstra uma tendência na evolução da linguagem para reduzir o código "cerimonial" e diminuir a barreira de entrada para iniciantes e para a criação de scripts simples. No entanto, por baixo dos panos, o compilador C# ainda gera essa estrutura clássica. Compreender a forma explícita com o método Main é fundamental para uma base sólida sobre como os programas C# são estruturados e executados.

Seção 2:

Manipulação de Dados: Variáveis e Tipos Primitivos

No cerne da lógica de programação está a capacidade de armazenar, manipular e recuperar dados. Em C#, isso é feito através de variáveis, que são essencialmente nomes simbólicos para locais de memória. Cada variável deve ser declarada com um tipo de dado específico, uma regra que define que tipo de valor ela pode armazenar (números, texto, etc.) e quais operações podem ser realizadas com ela. A escolha do tipo de dado correto é uma decisão fundamental que afeta a eficiência, a precisão e a correção do programa.

2.1 Tipos Numéricos Integrais (Números Inteiros)

Estes tipos são usados para representar números inteiros, sem componentes fracionários.

- **int:** É o alias em C# para o tipo System.Int32 da plataforma.NET. Este é o tipo de dado padrão e mais comumente usado para números inteiros. Ele utiliza 32 bits de memória para armazenar valores, o que lhe confere um intervalo de -2.147.483.648 a 2.147.483.647. É a escolha ideal para a maioria das situações que envolvem contagens, índices e números inteiros de magnitude moderada.
 - **Exemplo de declaração:** `$int idade = 25;$`
- **long:** É o alias para System.Int64. Este tipo utiliza 64 bits de memória e é usado para armazenar números inteiros que excedem o limite do int. Seu intervalo é vastamente maior, de -9.223.372.036.854.775.808 a 9.223.372.036.854.775.807. Ao atribuir um valor literal a uma variável long, é uma boa prática adicionar o sufixo L para indicar explicitamente ao compilador que o número é do tipo long.
 - **Exemplo de declaração:** `$long populacaoMundial = 8000000000L;$`

2.2 Tipos Numéricos de Ponto Flutuante (Números com Casas Decimais)

Esses tipos são usados para números que podem ter uma parte fracionária. A escolha entre eles é uma das decisões mais críticas para um programador, pois envolve um compromisso entre intervalo, precisão e a natureza da representação numérica.

- **double:** Alias para System.Double. É o tipo de ponto flutuante padrão em C#, utilizando 64 bits de memória. Oferece uma precisão de aproximadamente 15 a 16 dígitos significativos. O double é ideal para cálculos científicos, de engenharia ou qualquer cenário onde uma alta precisão é necessária, mas pequenas imprecisões de arredondamento inerentes à representação binária de frações são aceitáveis.

Exemplo de declaração: `$double altura = 1.82;$`

- **float:** Alias para System.Single. Utiliza 32 bits de memória, oferecendo uma precisão menor (cerca de 7 dígitos significativos) em comparação com o double. Seu uso é recomendado em situações onde a memória é uma restrição significativa, como em computação gráfica, ou quando a precisão do double é desnecessária. Literais do tipo float devem ser acompanhados do sufixo `f` ou `F`.

Exemplo de declaração: `$float peso = 75.5f;$`

- **decimal:** Alias para System.Decimal. Este é um tipo de 128 bits projetado especificamente para aplicações financeiras e monetárias. Ele oferece uma precisão altíssima, de 28 a 29 dígitos significativos, e, mais importante, é um tipo de ponto flutuante de base decimal, não binária.

A distinção entre a representação de base binária (float, double) e de base decimal (decimal) é fundamental. Tipos de base binária não conseguem representar com exatidão certas frações decimais, como 0.1. Isso significa que cálculos como `$0.1 + 0.2$` em um double não resultarão em exatamente `$0.3$`, mas em uma aproximação muito próxima. Para aplicações científicas, essa imprecisão é geralmente insignificante.

No entanto, em um sistema financeiro, onde cada centavo conta, esses erros de arredondamento podem se acumular e levar a discrepâncias inaceitáveis. O tipo decimal foi criado para resolver este problema, garantindo que valores decimais sejam armazenados e

calculados com exatidão. Literais do tipo decimal requerem o sufixo m ou M. * **Exemplo de declaração:** \$decimal salario = 2500.75m;\$

2.3 Tipos para Texto

Para manipular texto, C# oferece dois tipos de dados principais.

- **string:** Alias para System.String. É usado para representar sequências de caracteres, como palavras, frases ou qualquer texto. Os valores literais de string são delimitados por aspas duplas ("). Uma característica crucial do tipo string em C# é a sua **imutabilidade**. Isso significa que, uma vez que um objeto string é criado, seu conteúdo não pode ser alterado. Qualquer operação que pareça modificar uma string, como concatenar texto, na verdade cria um *novo* objeto string na memória com o resultado da operação. Essa característica tem implicações de desempenho importantes: concatenar strings repetidamente dentro de um laço pode ser ineficiente, pois gera muitas alocações de memória temporárias.

Exemplo de declaração: \$string nome = "Djalma";\$

- **char:** Alias para System.Char. É usado para representar um único caractere Unicode. Isso inclui não apenas letras e números, mas também uma vasta gama de símbolos. Os valores literais de char são delimitados por aspas simples (').

Exemplo de declaração: \$char inicial = 'D';\$

2.4 Tipo Lógico

Existe um único tipo para representar valores lógicos ou booleanos.

- **bool:** Alias para System.Boolean. Uma variável do tipo bool pode armazenar apenas um de dois valores possíveis: true (verdadeiro) ou false (falso). Este tipo é a espinha dorsal de toda a lógica de controle de fluxo, sendo utilizado em estruturas de decisão (if, switch) e laços de repetição (while, for) para determinar o caminho que a execução do programa deve seguir.

Exemplo de declaração: \$bool alunoAprovado = true;\$

A tabela a seguir consolida as informações dos tipos de dados numéricos primitivos, servindo como uma referência rápida para consulta.

Palavra-chave C#	Tipo.NET	Tamanho	Intervalo de Valores (Aproximado)	Uso Típico
int	System.Int32	32 bits	-2.1 bilhões a 2.1 bilhões	Padrão para números inteiros, contadores, índices.
long	System.Int64	64 bits	-9×10^{18} a 9×10^{18}	Números inteiros muito grandes, IDs de banco de dados, etc.
float	System.Single	32 bits	$\pm 1.5 \times 10^{-45}$ a $\pm 3.4 \times 10^{38}$	Gráficos, física de jogos, onde a performance é crítica.
double	System.Double	64 bits	$\pm 5.0 \times 10^{-324}$ a $\pm 1.7 \times 10^{308}$	Padrão para números decimais, cálculos científicos e de engenharia.
decimal	System.Decimal	128 bits	$\pm 1.0 \times 10^{-28}$ a $\pm 7.9 \times 10^{28}$	Cálculos financeiros, monetários e qualquer cenário que exija exatidão decimal.

Seção 3:

Operadores: As Ferramentas para Ações e Comparações

Operadores são símbolos especiais que realizam operações em variáveis e valores. Eles são os "verbos" da linguagem de programação, permitindo calcular, atribuir, comparar e combinar dados para formar expressões. Compreender como os operadores funcionam, e especialmente a ordem em que são avaliados, é fundamental para evitar erros lógicos.

3.1 Operadores Aritméticos

Os operadores aritméticos são usados para executar cálculos matemáticos. C# suporta os operadores padrão para as operações básicas.

- **+** (**Adição**): Soma dois operandos. Ex: `$int soma = 10 + 5;$` // soma será 15.
- **-** (**Subtração**): Subtrai o segundo operando do primeiro. Ex: `$int diferenca = 10 - 5;$` // diferenca será 5.
- **(Multiplicação)**: Multiplica dois operandos. Ex: `$int produto = 10 * 5;$` // produto será 50.
- **/** (**Divisão**): Divide o primeiro operando pelo segundo. Ex: `$double quociente = 10.0 / 4.0;$` // quociente será 2.5.
- **%** (**Módulo**): Retorna o resto da divisão inteira. Ex: `$int resto = 10 % 3;$` // resto será 1.

Uma particularidade importante é a **divisão de inteiros**. Quando ambos os operandos de uma divisão são do tipo `int`, o resultado também será um `int`, e qualquer parte fracionária é descartada (truncada). Por exemplo, a expressão `$5 / 2$` resultará em 2, não em 2.5. Este é um erro comum para iniciantes e deve ser observado com atenção. Para obter um resultado com casas decimais, pelo menos um dos operandos deve ser de um tipo de ponto flutuante (ex: `$5.0 / 2$`).

3.2 Operadores de Atribuição

Operadores de atribuição são usados para atribuir valores a variáveis.

- **=** (**Atribuição Simples**): O operador mais fundamental. Ele atribui o valor do operando à direita à variável à esquerda. Ex: `$int numero = 100;$`.
- **Operadores de Atribuição Composta**: São atalhos que combinam uma operação aritmética com uma atribuição. Eles tornam o código mais conciso e, para muitos desenvolvedores, mais legível, pois expressam a intenção de modificar uma variável "no local". A expressão `x = x + 5;` descreve uma operação matemática seguida de uma atribuição. A forma composta, `x += 5;`, descreve mais diretamente a intenção de "adicionar 5 a x". Essa clareza semântica melhora a legibilidade do código.
 - **+=**: Adição e atribuição. `$x += 5;$` é equivalente a `$x = x + 5;$`.
 - **-=**: Subtração e atribuição. `$x -= 5;$` é equivalente a `$x = x - 5;$`.
 - ***=**: Multiplicação e atribuição. `$x *= 5;$` é equivalente a `$x = x * 5;$`.

- **/=**: Divisão e atribuição. `$x /= 5;$` é equivalente a `$x = x / 5;$`.
- **%=**: Módulo e atribuição. `$x %= 5;$` é equivalente a `$x = x % 5;$`.

3.3 Operadores de Comparação (Relacionais)

Estes operadores comparam dois valores e o resultado da expressão é sempre um valor booleano (true ou false). Eles são a base para a tomada de decisões no código.

- **== (Igual a)**: Verifica se dois valores são iguais. Ex: `$(5 == 5)$` retorna true.
- **!= (Diferente de)**: Verifica se dois valores são diferentes. Ex: `$(5 != 5)$` retorna false.
- **> (Maior que)**: Verifica se o valor da esquerda é maior que o da direita. Ex: `$(10 > 5)$` retorna true.
- **< (Menor que)**: Verifica se o valor da esquerda é menor que o da direita. Ex: `$(10 < 5)$` retorna false.
- **>= (Maior ou igual a)**: Verifica se o valor da esquerda é maior ou igual ao da direita. Ex: `$(5 >= 5)$` retorna true.
- **<= (Menor ou igual a)**: Verifica se o valor da esquerda é menor ou igual ao da direita. Ex: `$(5 <= 5)$` retorna true.

É fundamental não confundir o operador de comparação `==` com o operador de atribuição `=`. Usar `=` em uma estrutura condicional (`if (x = 5)`) em vez de `==` (`if (x == 5)`) é um erro lógico comum que pode levar a bugs difíceis de diagnosticar.

3.4 Operadores Lógicos

Operadores lógicos são usados para combinar ou inverter expressões booleanas, permitindo a criação de condições complexas.

- **&& (E Lógico / AND)**: Retorna true somente se *ambas* as expressões à sua esquerda e direita forem true. Ex: `$(idade > 18 && possuiCNH == true)$`.
- **|| (OU Lógico / OR)**: Retorna true se *pelo menos uma* das expressões à sua esquerda ou direita for true. Ex: ``$(dia == "Sabado" |`
`| dia == "Domingo")$``.
- **! (NÃO Lógico / NOT)**: Inverte o valor de uma expressão booleana. Se a expressão é true, `!expressao` se torna false, e vice-versa. Ex: `!alunoAprovado`.

Os operadores `&&` e `||` em C# utilizam uma técnica chamada **avaliação de curto-circuito**. Para o operador `&&`, se a primeira expressão for `false`, o resultado geral já é `false`, então a segunda expressão nem sequer é avaliada.

Para o operador `||`, se a primeira expressão for `true`, o resultado geral já é `true`, e a segunda expressão também não é avaliada. Isso não só otimiza o desempenho, mas também pode ser usado para evitar erros, como tentar acessar uma propriedade de um objeto nulo.

Precedência de Operadores

A ordem em que os operadores são avaliados em uma expressão complexa não é simplesmente da esquerda para a direita. O compilador segue um conjunto de regras de precedência, similar às da matemática.

Por exemplo, a multiplicação (`*`) e a divisão (`/`) têm maior precedência do que a adição (`+`) e a subtração (`-`). Isso significa que na expressão `a = 5 + 3 * 2`, a multiplicação `3 * 2` é realizada primeiro, resultando em 6, e depois a soma `5 + 6`, atribuindo 11 a `a`.

A falta de compreensão dessas regras é uma fonte comum de erros lógicos. Para garantir a clareza e a correção, existe uma recomendação universal: **na dúvida, use parênteses**. Parênteses (`()`) forçam a ordem de avaliação desejada. A expressão `a = (5 + 3) * 2`; resultará em 16, pois a soma dentro dos parênteses é avaliada primeiro.

Seção 4:

Controle do Fluxo de Execução: Tomada de Decisões e Repetições

O verdadeiro poder de um programa reside em sua capacidade de não apenas executar instruções em sequência, mas de analisar dados e alterar seu comportamento com base neles. Isso é alcançado através de estruturas de controle de fluxo, que permitem ao programa tomar decisões (estruturas condicionais) e repetir tarefas (laços de repetição).

4.1 Estruturas Condicionais

Estruturas condicionais executam diferentes blocos de código dependendo se uma condição especificada é verdadeira ou falsa.

- **if, else, else if:** Esta é a estrutura de decisão mais fundamental.
 - ✓ O bloco if executa seu código se a condição booleana entre parênteses for true.
 - ✓ O bloco else, que é opcional, executa seu código se a condição do if for false.
 - ✓ O bloco else if permite encadear múltiplas condições. Se o primeiro if for falso, a condição do else if é testada. Pode-se ter vários else if em sequência, e o primeiro que avaliar para true terá seu bloco de código executado.

Exemplo Prático:

```
Console.WriteLine("Digite sua idade:");  
int idade = Convert.ToInt32(Console.ReadLine());
```

```
if (idade < 18)  
{  
    Console.WriteLine("Você é menor de idade.");  
}  
else if (idade >= 18 && idade < 65)  
{  
    Console.WriteLine("Você é um adulto.");  
}  
else  
{  
    Console.WriteLine("Você é um idoso.");  
}
```

- **switch, case, default:** A estrutura switch é uma alternativa ao if-else if-else que pode ser mais legível quando se está comparando o valor de uma única variável contra uma série de valores constantes.
 - switch (variavel): Inicia a estrutura, especificando a variável a ser testada.
 - case valor:: Define um rótulo para um valor específico. Se a variável corresponder a

este valor, o código abaixo dele é executado.

- **break;:** É obrigatório ao final de cada bloco case (que contém código). Ele encerra a execução do switch e impede que o controle "caia" para o próximo case.
- **default;:** É opcional e funciona como o else final. Seu código é executado se nenhum dos case anteriores corresponder ao valor da variável.

A exigência do break em C# é uma decisão de design deliberada. Em outras linguagens como C++, a ausência de um break causa um "fall-through" implícito, onde a execução continua no próximo case, uma fonte comum de bugs. C# força o programador a ser explícito sobre o fluxo de controle, tornando o código mais seguro e previsível por padrão.

Exemplo Prático:

```
Console.WriteLine("Digite um número de 1 a 3:");
int opcao = Convert.ToInt32(Console.ReadLine());

switch (opcao)
{
    case 1:
        Console.WriteLine("Você escolheu a opção 1.");
        break;
    case 2:
        Console.WriteLine("Você escolheu a opção 2.");
        break;
    case 3:
        Console.WriteLine("Você escolheu a opção 3.");
        break;
    default:
        Console.WriteLine("Opção inválida.");
        break;
}
```

4.2 Laços de Repetição (Iteração)

Laços, ou loops, são usados para executar um bloco de código repetidamente até que uma condição de parada seja satisfeita.

- **for:** O laço for é ideal quando o número de repetições é conhecido de antemão. Sua sintaxe é composta por três partes, separadas por ponto e vírgula, dentro dos parênteses:
 1. **Inicializador:** Executado uma única vez antes do início do laço. Geralmente, declara e inicializa uma variável de controle (ex: `int i = 0`).
 2. **Condição:** Uma expressão booleana avaliada antes de cada iteração. Enquanto `for true`, o laço continua. (ex: `i < 10`).
 3. **Iterador:** Executado ao final de cada iteração. Geralmente, incrementa ou decrementa a variável de controle (ex: `i++`).

Exemplo Prático:

```
// Imprime os números de 0 a 9
for (int i = 0; i < 10; i++)
{
    Console.WriteLine(i);
}
```

- **while:** O laço while executa um bloco de código repetidamente *enquanto* uma condição booleana permanecer true. A condição é testada *antes* de cada iteração. Se a condição for falsa desde o início, o bloco de código nunca será executado. É ideal para situações onde o número de iterações não é conhecido previamente.

Exemplo Prático:

```
string senha = "";
while (senha!= "1234")
{
    Console.Write("Digite a senha: ");
    senha = Console.ReadLine();
}
Console.WriteLine("Acesso concedido!");
```


- **do-while:** O laço do-while é semelhante ao while, com uma diferença crucial: a condição é testada *após* a execução do bloco de código. Isso garante que o código dentro do laço seja executado **pelo menos uma vez**, independentemente da condição ser verdadeira ou falsa. Esta estrutura é a solução ideal para cenários de "executar e depois validar", como a validação de entrada do usuário. Usar um do-while para este padrão evita a duplicação de código que seria necessária com um laço while (onde a solicitação de entrada teria que ser feita antes do laço e novamente dentro dele).

Exemplo Prático:

```
int numero;  
do  
{  
    Console.Write("Digite um número positivo: ");  
    numero = Convert.ToInt32(Console.ReadLine());  
} while (numero <= 0);  
Console.WriteLine("Número válido inserido: " + numero);
```

4.3 Comandos de Salto (Jump Statements)

Comandos de salto alteram incondicionalmente o fluxo de controle dentro de um laço ou switch.

- **break:** Termina imediatamente o laço (for, while, do-while) ou a estrutura switch mais interna em que se encontra. O controle do programa passa para a primeira instrução após o laço ou switch terminado.

Exemplo Prático:

```
for (int i = 0; i < 100; i++)  
{  
    if (i == 5)  
    {  
        break; // O laço termina quando i for 5  
    }  
    Console.WriteLine(i); // Imprime de 0 a 4  
}
```

- **continue:** Pula o restante do código na iteração atual e força o início da próxima iteração do laço. Diferente do break, o laço não é terminado.

Exemplo Prático:

```
for (int i = 0; i < 10; i++)  
{  
    if (i % 2 != 0) // Se i for ímpar
```

```
{  
    continue; // Pula para a próxima iteração  
}  
Console.WriteLine(i); // Imprime apenas os números pares: 0, 2, 4, 6, 8  
}
```

Seção 5:

Interação com o Usuário: Entrada e Saída no Console

Aplicações de console vivem da interação com o usuário. Em C#, a classe `Console` fornece os métodos essenciais para exibir informações na tela (saída) e para capturar dados digitados pelo usuário no teclado (entrada). Dominar esses comandos é o primeiro passo para criar programas interativos.

5.1 Saída de Dados

Para exibir texto, resultados de cálculos ou mensagens para o usuário, utilizam-se principalmente dois métodos.

- **`Console.WriteLine()`**: Este método escreve a string ou o valor especificado na janela do console, e em seguida, **move o cursor para o início da próxima linha**. O "Line" em `WriteLine` refere-se a essa quebra de linha automática. É o método mais comum para exibir mensagens completas ou resultados.
- **`Console.Write()`**: Este método escreve a string ou o valor especificado na janela do console, mas **mantém o cursor na mesma linha**, imediatamente após o último caractere escrito. É particularmente útil para criar prompts de entrada, onde se espera que o usuário digite na mesma linha da pergunta.

Exemplo Comparativo:

O código a seguir demonstra a diferença fundamental entre os dois métodos.

```
// Usando Console.Write()  
Console.Write("Olá, ");  
Console.Write("Mundo!");  
// Saída: Olá, Mundo! (e o cursor fica após o '!')
```

```
Console.WriteLine("\n---"); // Adiciona uma linha de separação
```

```
// Usando Console.WriteLine()
Console.WriteLine("Olá,");
Console.WriteLine("Mundo!");
// Saída:
// Olá,
// Mundo! (e o cursor fica em uma nova linha abaixo)
```

5.2 Entrada de Dados

Para capturar dados que o usuário digita no teclado, o método principal é `Console.ReadLine()`.

- **`Console.ReadLine()`**: Este método pausa a execução do programa e aguarda o usuário digitar uma sequência de caracteres. A leitura é concluída quando o usuário pressiona a tecla **Enter**. O método então retorna todos os caracteres digitados como um valor do tipo **string**.

É absolutamente crucial entender que `Console.ReadLine()` **sempre** retorna uma string, independentemente do que o usuário tenha digitado. Se o usuário digitar "25", o método não retorna o número 25, mas sim a string "25". Este fato introduz a necessidade de conversão de tipos.

Conversão de Tipos (Type Casting)

O fato de `Console.ReadLine()` retornar exclusivamente uma string é uma manifestação direta do sistema de tipos forte do C#. A linguagem exige que o programador seja explícito sobre suas intenções ao manipular dados de tipos diferentes. Não há uma conversão automática (implícita) de uma string "123" para o número inteiro 123.

Essa necessidade de conversão explícita não deve ser vista como um obstáculo, mas como a primeira experiência prática do aluno com a segurança de tipos, uma das principais características do C# que ajuda a prevenir erros.

Para utilizar a entrada do usuário em operações matemáticas ou lógicas, é necessário

converter a string retornada para o tipo de dado apropriado. A classe `Convert` fornece um conjunto de métodos para essa finalidade.

- **`Convert.ToInt32()`**: Converte uma representação de string de um número para um `int` (inteiro de 32 bits).
- **`Convert.ToDouble()`**: Converte uma representação de string de um número para um `double`.
- **`Convert.ToDecimal()`**: Converte uma representação de string de um número para um `decimal`.
- **`Convert.ToBoolean()`**: Converte uma representação de string de um valor booleano (como "true" ou "false") para um `bool`.

Exemplo Prático de Leitura e Conversão:

O programa a seguir solicita o nome e a idade do usuário, demonstrando a leitura de uma string e a conversão para um inteiro.

```
// Solicita o nome (não precisa de conversão)
```

```
Console.Write("Digite seu nome: ");
```

```
string nome = Console.ReadLine();
```

```
// Solicita a idade (precisa de conversão)
```

```
Console.Write("Digite sua idade: ");
```

```
string idadeString = Console.ReadLine(); // Lê como string
```

```
int idade = Convert.ToInt32(idadeString); // Converte para int
```

```
// Exibe os dados processados
```

```
Console.WriteLine($"Olá, {nome}! No próximo ano, você terá {idade + 1} anos.");
```

Neste exemplo, se a conversão de `idadeString` para `int` não fosse realizada, a operação `idade + 1` resultaria em um erro de compilação, pois não é possível somar um número a uma string dessa maneira.

Seção 6:

Estruturas de Dados Básicas: Arrays Unidimensionais

À medida que os programas se tornam mais complexos, surge a necessidade de trabalhar com coleções de dados, em vez de apenas variáveis individuais. A estrutura de dados mais fundamental para este propósito é o **array**. Um array é uma coleção de tamanho fixo que armazena uma sequência de elementos, todos do mesmo tipo de dado.

6.1 Declaração e Inicialização de Arrays

Para usar um array, ele deve primeiro ser declarado e depois inicializado (ou seja, a memória para ele deve ser alocada).

- **Declaração:** A sintaxe para declarar um array consiste em especificar o tipo dos elementos seguido por colchetes `` e o nome da variável.
 - Exemplo: `$int numeros;$`
- **Inicialização:** A inicialização é feita com a palavra-chave `new`, seguida do tipo, e o tamanho do array entre colchetes. Isso aloca um bloco contíguo de memória para armazenar os elementos.
 - Exemplo: `$numeros = new int;$` // Cria um array para 5 inteiros.

Quando um array é inicializado desta forma, seus elementos recebem o valor padrão para o seu tipo (por exemplo, 0 para `int`, `false` para `bool`, e `null` para strings).

É possível declarar e inicializar um array em uma única linha. Além disso, pode-se fornecer os valores iniciais dos elementos diretamente, o que também define o tamanho do array.

Exemplos de Declaração e Inicialização:

```
// 1. Declarar e inicializar um array de 5 inteiros.
```

```
// Os elementos serão inicializados com o valor padrão 0.
```

```
int arrayDeInteiros = new int;
```

```
// 2. Declarar e inicializar um array de strings com valores pré-definidos.
```

```
// O tamanho do array (3) é inferido a partir do número de elementos.
```

```
string nomes = new string { "Ana", "Beto", "Carlos" };
```

```
// 3. Sintaxe simplificada para inicialização com valores pré-definidos.  
double notas = {9.5, 8.0, 7.2, 10.0};
```

6.2 Acesso a Elementos

Os elementos de um array são acessados através de um **índice**. Em C#, os arrays são **baseados em zero**, o que significa que o índice do primeiro elemento é 0, o do segundo é 1, e assim por diante, até o último elemento, que tem o índice tamanho - 1.

Tentar acessar um elemento com um índice que está fora dos limites válidos do array (menor que 0 ou maior ou igual ao seu tamanho) resultará em um erro em tempo de execução chamado `IndexOutOfRangeException`. Este é um dos erros mais comuns ao se trabalhar com arrays e destaca a importância de controlar cuidadosamente os índices.

Exemplo de Acesso e Modificação:

```
string nomes = new string { "Ana", "Beto", "Carlos" };
```

```
// Acessando para leitura
```

```
Console.WriteLine("O primeiro nome é: " + nomes); // Saída: O primeiro nome é: Ana
```

```
// Acessando para modificação (escrita)
```

```
nomes = "Beatriz"; // Altera o segundo elemento de "Beto" para "Beatriz"
```

```
Console.WriteLine("O segundo nome agora é: " + nomes); // Saída: O segundo nome agora é:  
Beatriz
```

6.3 A Propriedade Length

Todo array em C# possui uma propriedade chamada Length, que retorna o número total de elementos que o array pode conter. Esta propriedade é somente leitura (seu valor não pode ser alterado) e é extremamente útil para controlar laços de repetição que precisam percorrer todos os elementos do array, garantindo que o código não tente acessar um índice inválido.

Exemplo de uso da propriedade Length:

```
double notas = { 9.5, 8.0, 7.2, 10.0 };
```

```
Console.WriteLine("O array de notas contém " + notas.Length + " elementos."); // Saída: 4
```

6.4 Percorrendo Arrays com Laços

A combinação de arrays e laços de repetição é uma das técnicas mais poderosas na programação procedural. Existem duas formas principais de percorrer (ou iterar sobre) os elementos de um array em C#.

- **Laço for:** O laço for é a maneira tradicional e mais flexível de percorrer um array. Ele utiliza uma variável de controle como índice, dando acesso direto a cada posição do array. Isso permite não apenas ler os valores, mas também modificá-los, ou até mesmo percorrer o array em ordens diferentes (por exemplo, de trás para frente).
- **Exemplo com for:**

```
int numeros = { 10, 20, 30, 40, 50 };  
for (int i = 0; i < numeros.Length; i++)  
{  
    Console.WriteLine($"O elemento no índice {i} é {numeros[i]}");  
}
```
- **Laço foreach:** O laço foreach oferece uma sintaxe mais simples e legível para percorrer todos os elementos de uma coleção, como um array, do início ao fim. Ele abstrai o gerenciamento do índice, tornando o código menos propenso a erros de "off-by-one" (erros de lógica com o índice). A cada iteração, uma variável temporária recebe o valor do

elemento atual da coleção.

A existência de dois laços para percorrer arrays não é uma redundância, mas um reflexo de um trade-off entre controle e simplicidade. O `foreach` é mais expressivo da intenção de "fazer algo para cada item" e deve ser a escolha padrão para iterações simples e de somente leitura. O laço `for` é a ferramenta mais poderosa, a ser utilizada quando o controle fino sobre o índice é necessário para a lógica do programa.

Exemplo com `foreach`:

```
csharp int numeros = { 10, 20, 30, 40, 50 }; Console.WriteLine("Percorrendo com  
foreach:"); foreach (int numero in numeros) { Console.WriteLine(numero); }
```

Conclusão

Este glossário apresentou os pilares fundamentais da programação estruturada em C# para o ambiente de console. A jornada começou com a desmistificação da estrutura básica de um programa, explicando o propósito de cada diretiva e bloco de código que forma o esqueleto de uma aplicação. Em seguida, explorou-se a manipulação de dados através de variáveis e a importância da escolha correta entre os diversos tipos de dados primitivos, um conceito central para a escrita de código eficiente e preciso.

Foram detalhados os operadores aritméticos, de atribuição, de comparação e lógicos — as ferramentas que permitem a construção de expressões e a implementação da lógica de negócio. O controle do fluxo de execução foi abordado através das estruturas condicionais `if-else` e `switch-case`, que capacitam o programa a tomar decisões, e dos laços de repetição `for`, `while` e `do-while`, que permitem a execução iterativa de tarefas.

A interação com o usuário, um aspecto vital das aplicações de console, foi coberta com os métodos de entrada e saída, com ênfase na necessidade crucial da conversão de tipos. Finalmente, foi introduzida a primeira estrutura de dados, o array unidimensional, juntamente com as técnicas para sua manipulação e iteração.

O domínio destes conceitos — estrutura, manipulação de dados, operadores, controle de fluxo e coleções básicas — constitui a base indispensável sobre a qual todo o conhecimento subsequente em programação é construído.

Para os alunos que iniciam sua formação em desenvolvimento de sistemas, a consolidação destes fundamentos é o passo mais crítico antes de avançar para paradigmas mais

complexos, como a Orientação a Objetos. A prática contínua com estes comandos em C# não apenas solidificará a compreensão da sintaxe da linguagem, mas, mais importante, desenvolverá o raciocínio lógico que é a verdadeira essência da programação.